TAFS – Meezan Bank Bill Collection API Integration Audit & Reference Document

TAFS Development Team

8 June 2026

Purpose of This Document

This document supersedes the previously shared integration guide and is prepared in direct response to the points raised by the Meezan Bank IT Team. It provides:

- Full request and response payload samples for **Bill Inquiry** and **Bill Payment**, covering both **success** and **failure** scenarios.
- A clear explanation that the fields returned for an **unsuccessful Bill Inquiry** are **structurally different** from those returned for a **successful Bill Inquiry** (different field names entirely – not just different values).
- A consolidated table of all **API response codes** used across both services.
- Direct clarifications against each point raised by the IT team.

This audit is based directly on the production source code of the integration (`meezan.service.ts` / `meezan.controller.ts`), so every payload shown here is guaranteed to match what the live system sends and receives.

1 Base URL & Authentication

1.1 Base URL

`https://tafs-backend-production.up.railway.app/api/v1/meezan`

1.2 Authentication Fields

Every request (both Bill Inquiry and Bill Payment) must carry the following three authentication fields in the JSON body. These are validated on *every* call before any business logic executes:

Field	Type	Description
`ServiceUserId`	string	Service authentication ID issued to Meezan
`UserPassword`	string	Password associated with the service user ID
`BillCompanyCode`	string	Company / campus identifier (`TAFCS`)

Validation order:

1. `ServiceUserId` and `UserPassword` are checked first. If either is wrong, the request is rejected with `ResponseCode "094"`.
2. If `BillCompanyCode` is supplied and does not match, the request is rejected with `ResponseCode "095"`.
3. Only after both checks pass does the system proceed to look up the voucher.

2 Service 1 – Bill Inquiry

Endpoint: `POST /bill-inquiry` **HTTP Status:** Always `200 OK`
The HTTP status code is always 200; the actual outcome (success or failure) is communicated through the JSON body – see Section 2.4 for why this matters.

2.1 Request Payload

```
{
  "ServiceUserId": "0b986d818b54b6eeedbf7658",
  "UserPassword": "e67f36b15bd1d034017354db2486a82b",
  "BillCompanyCode": "TAFCS",
  "VoucherNumber": "05260003357"
}
```

Field	Type	Required	Description
`ServiceUserId`	string	Yes	Authentication ID
`UserPassword`	string	Yes	Authentication password
`BillCompanyCode`	string	Yes	Company / campus identifier
`VoucherNumber`	string	Yes	Voucher number to look up

2.2 Success Response – Voucher Found and Unpaid

HTTP 200

```
{
  "StatusCode": "00",
  "StatusDesc": "Unpaid",
  "student_name": "TEST STUDENT B",
  "student_id": "2",
  "due_date": "20260521",
  "Amount_WID_Date": "50000",
  "Amount_AD_Date": "51000",
  "BillingMonth": "2504",
  "Remarks": "Late payment surcharge applies"
}
```

Field	Description
`StatusCode`	`"00"` – indicates a successful lookup
`StatusDesc`	`"Unpaid"` – the voucher exists and is payable
`student_name`	Full name of the student
`student_id`	Student CC (registration) number
`due_date`	Due date, format `YYYYMMDD`
`Amount_WID_Date`	Amount payable *within* (on or before) due date, in PKR
`Amount_AD_Date`	Amount payable *after* due date (includes late surcharge), in PKR
`BillingMonth`	Billing period, format `YYMM`
`Remarks`	`"Surcharge waived"`, `"Late payment surcharge applies"`, or `""`

Note: `Remarks` is set to `"Surcharge waived"` when the voucher has been manually exempted from late fees, `"Late payment surcharge applies"` when the inquiry is being made after the due date, and an empty string `""` otherwise.

2.3 Failure Response – Voucher Not Found / Invalid / Void / Paid / Expired / Auth Error

HTTP 200 (the transport-level status remains `200`; failure is indicated purely by the body)

```
{
  "ResponseCode": "091",
  "ResponseDesc": "Voucher Id is invalid"
}
```

Field	Description
`ResponseCode`	3-digit numeric code identifying the failure reason (see Section 4)
`ResponseDesc`	Human-readable description of the failure

2.4 IMPORTANT – Field Differences Between Success and Failure Responses

This is the point specifically raised by the IT team and must be handled carefully on the bank's parsing side: **a successful Bill Inquiry response and a failed/erroneous Bill Inquiry response do not share the same field names.** They are two structurally distinct payload shapes returned from the same endpoint:

Successful Inquiry – fields present	Failed Inquiry – fields present
`StatusCode`	`ResponseCode`
`StatusDesc`	`ResponseDesc`
`student_name`	*(none of the voucher/student fields are present)*
`student_id`	
`due_date`	
`Amount_WID_Date`	
`Amount_AD_Date`	
`BillingMonth`	
`Remarks`	

Practical guidance for the bank's integration layer:

- Do *not* assume the voucher/student fields (`student_name`, `due_date`, etc.) will always be present – they are *only* present when `StatusCode == "00"`.
- The presence of the `ResponseCode` key (rather than `StatusCode`) is itself the signal that the inquiry failed.
- Recommended parsing logic: check for `StatusCode` first – if present and equal to `"00"`, treat as success and read the voucher fields; otherwise, read `ResponseCode`/`ResponseDesc` and treat as a failure using the table in Section 4.

3 Service 2 – Bill Payment

Endpoint: `POST /bill-payment` **HTTP Status:** Always `200 OK`

3.1 Request Payload


```
{
  "ServiceUserId": "0b986d818b54b6eeedbf7658",
  "UserPassword": "e67f36b15bd1d034017354db2486a82b",
  "BillCompanyCode": "TAFCS",
  "VoucherNumber": "05260003357",
  "TransDate": "20260512",
  "TransAuthenticationCode": "AUTH123456",
  "TransAmount": "50000",
  "PaymentMode": "ONLINE",
  "BankCode": "MEZN",
  "BankName": "Meezan Bank Limited",
  "ChequeNo": "",
  "Status": "C",
  "ReasonCode": "",
  "DateOfReturn": "",
  "ReasonDescription": ""
}
```


Field	Type	Required	Description
`ServiceUserId`	string	Yes	Authentication ID
`UserPassword`	string	Yes	Authentication password
`BillCompanyCode`	string	No*	Company / campus identifier
`VoucherNumber`	string	Yes	Voucher number being paid
`TransDate`	string	Yes	Transaction date, format `YYYYMMDD`
`TransAuthenticationCode`	string	Yes	Bank transaction reference / auth code
`TransAmount`	string	Yes	Amount paid, in PKR
`PaymentMode`	string	Yes	Payment mode (e.g. `CASH`, `CHQ`, `ONLINE`)
`BankCode`	string	Yes	Bank's branch / routing code
`BankName`	string	Yes	Name of the bank / branch
`ChequeNo`	string	No	Cheque number (cheque payments only)
`Status`	string	Yes	`C` = Cleared, `L` = Lodged, `R` = Returned
`ReasonCode`	string	No	Reason code (used when `Status = R`)
`DateOfReturn`	string	No	Return date, format `YYYYMMDD` (used when `Status = R`)
`ReasonDescription`	string	No	Reason description (used when `Status = R`)

* `BillCompanyCode` is optional in the payment request body; however, if it *is* supplied, it is still validated against our records and a mismatch returns `ResponseCode "095"`.

3.2 Status Field – Behaviour

Status	Meaning	System action
`C`	Cleared – payment confirmed	Voucher marked `PAID`; deposit created; fee heads and (if applicable) overdue surcharges allocated and settled
`L`	Lodged – payment pending	No ledger update; logged only
`R`	Returned – payment rejected	No ledger update; reason logged only

Only `Status = "C"` triggers actual ledger posting (deposit creation, fee allocation, surcharge

settlement, and marking the voucher `PAID`). `L` and `R` are acknowledged with a `StatusCode "00"` response but make *no* change to the voucher or student ledger.

3.3 Success Response – Payment Posted (`Status = "C"`)

HTTP 200

```
{
  "StatusCode": "00",
  "StatusDesc": "Success"
}
```

This confirms the voucher has been marked `PAID`, a deposit record has been created, and all applicable fee heads (and overdue surcharges, where relevant) have been settled.

3.4 Acknowledged but Not Posted – Lodged / Returned (`Status = "L"` or `"R"`)

HTTP 200

```
{
  "StatusCode": "00",
  "StatusDesc": "Lodged/Returned \xe2\x80\x94 not posted"
}
```

This is *not* an error – `StatusCode "00"` confirms the request was received and acknowledged. It simply means no ledger posting occurred because the transaction was not in a `Cleared` state. The reason (for `Status = "R"`, including `ReasonCode`/`ReasonDescription`) is recorded in our internal logs for audit purposes.

3.5 Failure Response – Voucher Not Found / Already Paid / Expired / Auth Error / Exception

HTTP 200

```
{
  "ResponseCode": "097",
  "ResponseDesc": "Voucher already paid"
}
```

As with Bill Inquiry, failure responses use the `ResponseCode`/`ResponseDesc` field pair rather than `StatusCode`/`StatusDesc`, and carry no payment/voucher detail fields.

4 Consolidated API Response Code Reference

The following codes are returned (in the `ResponseCode` field, for failure cases) or as `StatusCode "00"` for success/acknowledgement cases, across **both** Bill Inquiry and Bill Payment:

Code	Description	Returned by / Meaning
00 (StatusCode)	Success / Unpaid / Acknowledged	Both services. For Inquiry: voucher found and unpaid. For Payment: payment posted (Status=C), or acknowledged without posting (Status=L/R).
091	Voucher Id is invalid	Both services. No voucher exists for the supplied VoucherNumber (including the 11-digit fallback lookup).
092	Voucher date is expired	Both services. The voucher's validity_date has passed relative to the inquiry/transaction date.
093	Voucher is void	Bill Inquiry only. The voucher has been administratively voided.
094	User invalid	Both services. ServiceUserId or UserPassword did not match our records.
095	Company code mismatch	Both services. BillCompanyCode was supplied but did not match our records.
096	General exception	Both services. An unexpected server-side error occurred while processing the request.
097	Voucher is already paid / Voucher already paid	Both services. The voucher's status is already PAID.

Notes on code usage:

- Codes 091, 092, 094, 095, 096, and 097 are common to both Bill Inquiry and Bill Payment.
- Code 093 (*Voucher is void*) is returned **only** by Bill Inquiry. Bill Payment does not currently perform a separate void check (a voided voucher would typically already be reflected through one of the other statuses).
- All response codes are returned with **HTTP 200** – the bank's integration layer must inspect the JSON body (StatusCode vs. ResponseCode) to determine the outcome, not the HTTP transport status.

5 Voucher Number Resolution Logic

For both services, the voucher is first looked up by an exact match on voucher_number. If no match is found *and* the supplied VoucherNumber is exactly 11 characters long, the system attempts a fallback lookup by treating the last 7 characters as a numeric internal voucher ID (VoucherNumber.slice(4)). This allows older/short-format voucher numbers issued by the bank to still resolve correctly. If neither lookup succeeds, ResponseCode "091" (*Voucher Id is invalid*) is returned.

6 Overdue Surcharge Logic

- `Amount_WID_Date` – payable amount if settled *on or before* the due date.
- `Amount_AD_Date` – payable amount if settled *after* the due date (includes the applicable late-payment surcharge).
- Whether a transaction is treated as overdue is determined by comparing the transaction/inquiry date to the voucher's `due_date` (both normalized to midnight, so same-day payments are *not* considered overdue).
- If a voucher has been manually marked as `surcharge_waived`, no surcharge is applied regardless of the date, and `Remarks` will read `"Surcharge waived"`.
- During Bill Payment with `Status = "C"` on an overdue transaction, any un-waived arrear surcharges linked to the voucher are automatically settled alongside the regular fee heads in the same transaction.

7 Clarifications on Points Raised by the IT Team

1. **"The previous document does not include API request and response samples."**
   Resolved – Sections 2.1, 2.2, 2.3, 3.1, 3.3, 3.4 of this document now provide complete, code-accurate JSON samples for every request and response variant (success and failure) of both Bill Inquiry and Bill Payment.
2. **"The API response codes are not mentioned."**
   Resolved – Section 4 provides a single consolidated table of every response code (`00`, `091`–`097`) returned by either service, with its meaning and which service(s) return it.
3. **"The fields received for an unsuccessful Inquiry are different from those in a successful Inquiry."**
   Confirmed and documented in detail in Section 2.4. This is by design: a successful inquiry returns voucher/student details under `StatusCode`/`StatusDesc` plus the amount and billing fields, whereas a failed inquiry returns only `ResponseCode`/`ResponseDesc` with no voucher data. We recommend the bank's parser branch on the presence of `StatusCode == "00"` to distinguish the two shapes, as detailed in Section 2.4.

8 Recommended Integration Flow

1. `POST /bill-inquiry` with `VoucherNumber`.
2. Check whether the response contains `StatusCode == "00"`:
   - If yes – voucher is valid and unpaid; compare today's date against `due_date` to decide whether to use `Amount_WID_Date` or `Amount_AD_Date`.
   - If no – read `ResponseCode`/`ResponseDesc` and handle per Section 4 (e.g. do not proceed to payment if the voucher is invalid, void, expired, or already paid).
3. `POST /bill-payment` with `VoucherNumber`, the correct `TransAmount`, and `Status = "C"`.
4. Check the response: `StatusCode == "00"` with `StatusDesc == "Success"` confirms posting; `StatusDesc == "Lodged/Returned -- not posted"` confirms acknowledgement without posting; a `ResponseCode` indicates a failure to be handled per Section 4.